Programación en Móviles

Laboratorio 12
Cine Castillo

Tema: Aplicación de películas con autenticación, API REST, almacenamiento local y arquitectura MVVM.

Estudiante: Jesús Castillo Sumire
Sección: C24D
Docente: Juan José León Suiyon
Fecha: 11 de noviembre de 2026
Modalidad: Trabajo individual

Presentación

Desarrollé Cine Castillo para consultar películas, revisar su información y conservar favoritas en el dispositivo. Integré Firebase para identificar a los usuarios, Firestore para distinguir sus roles y TMDB para obtener información real de películas. La copia local permite seguir consultando el catálogo cuando se pierde la conexión.

La entrega está organizada en el repositorio del curso, en Semana_12/CineCastillo. Se mantiene la fase base en main y la versión mejorada en mejora-ia. Se eligió TMDB siguiendo la indicación del docente de utilizar una API más completa que los ejemplos básicos de la guía.

Objetivo del trabajo

Conectar las distintas capas de una app Android sin mezclar la interfaz con el acceso a datos: autenticar, descargar, guardar, mostrar y recuperar información ante fallos de conexión. Además, comprobar su comportamiento con las 14 pruebas de aceptación de la guía.

Resultado de la verificación: 14 casos de aceptación aprobados en el emulador y 8 pruebas unitarias aprobadas. Las evidencias se incluyen después de la explicación del código. La versión documentada es 1.0.2.

1. Estructura del proyecto

Se muestra el código de la aplicación y su organización funcional. Se omiten archivos de configuración, archivos generados y credenciales.

```
com.tecsup.cinecastillo/
  MainActivity.kt / CineApplication.kt / AppContainer.kt
  SyncNotifications.kt
  data/
    auth/          AuthRepository.kt, FirebaseAuthRepository.kt
    remote/        MovieApi.kt, RetrofitClient.kt, dto/
    local/         MovieEntity.kt, MovieDao.kt, MovieDatabase.kt
    repository/    MovieRepository.kt, MovieRepositoryImpl.kt
    mapper/        Mappers.kt
  domain/model/    Movie.kt, Sesion.kt
  notification/    Notificador.kt, NotificadorAndroid.kt
  ui/
    UiState.kt
    navigation/    AppNav.kt
    login/         LoginScreen.kt, AuthViewModel.kt
    lista/         CatalogScreen.kt, MovieListViewModel.kt
    detalle/       DetailScreen.kt, MovieDetailViewModel.kt
    favoritos/     FavoritosScreen.kt, FavoritosViewModel.kt
    admin/         AdminScreen.kt, AdminViewModel.kt
    components/    Cargando.kt, ErrorView.kt
    theme/         CineTheme.kt
    CreditsScreen.kt
  test/            MovieListViewModelTest.kt, AuthViewModelTest.kt
```

data: comunica la app con Firebase, TMDB y Room. **domain/model:** describe la película y la sesión que utiliza el resto del código. **ui:** contiene pantallas, navegación y ViewModel. **notification:** separa la solicitud de un aviso de su implementación Android.

Movie es la adaptación del modelo genérico Item al tema de películas. MovieApi cumple la función de ApiService, y RoomMovieRepository es la implementación del contrato MovieRepository. Los nombres describen el tema; las responsabilidades y los métodos exigidos se mantienen.

2. Herramientas utilizadas

Android Studio, Kotlin y Jetpack Compose para la aplicación; ViewModel, StateFlow y corrutinas para manejar estados; Retrofit y Gson para HTTP/JSON; Room para SQLite; Coil para carteles; Firebase Authentication y Cloud Firestore para acceso y roles; JUnit y repositorios falsos para pruebas; Git y GitHub para avances y entregables.

3. Flujo de la aplicación

1. Acceso: se comprueba la sesión actual. Si no existe, se muestra el formulario. Una instalación nueva sin red muestra un error con Reintentar; al recuperar Internet permite continuar al login.

2. Identificación: Firebase valida correo y contraseña. Firestore entrega el campo rol de usuarios/{uid}. El usuario normal ve catálogo, favoritas y créditos; admin también ve el panel de administración.

3. Sincronización: el ViewModel solicita refrescar al repositorio. Retrofit consulta discover/movie; el repositorio transforma DTO a Entity y guarda todo en una transacción Room.

4. Presentación: el DAO emite las películas guardadas mediante Flow. El ViewModel publica UiState y Compose lo observa respetando el ciclo de vida. La pantalla muestra Cargando, Error, Vacío o datos.

5. Detalle y favoritos: al tocar una película se pasa su ID a la ruta detail/{id}. El segundo endpoint movie/{id} completa la información local. Marcar un corazón modifica Room y actualiza las pantallas que observan esos datos.

6. Administración: el panel muestra totales, permite sincronizar y pide confirmación antes de vaciar la copia local. Tras eliminarla, catálogo y contadores se actualizan juntos.

7. Notificación y salida: una sincronización correcta genera “Se guardaron N elementos”, si hay permiso. Tocar el aviso abre la app. Cerrar sesión devuelve al formulario y la siguiente apertura vuelve a pedir acceso.

Por qué se utiliza una copia local

La pantalla siempre observa Room. No recibe una lista remota independiente que luego pueda diferir de Favoritos. Esto permite que una misma película conserve su estado y se pueda mostrar cuando falla Internet. La autenticación inicial necesita conexión; los favoritos se guardan en este dispositivo, no se sincronizan entre cuentas o teléfonos.

Qué mejoró en la fase asistida

Se añadieron búsqueda y orden local, pruebas de comportamiento y un registro de indicaciones. La revisión final separó navegación y ViewModel por pantalla, unificó UiState, corrigió el espacio del catálogo en horizontal y completó el arranque sin red con reintento. El contenedor vive en Application para no reconstruirse al girar.

4. Validación antes de autenticar

Archivo: ui/login/AuthViewModel.kt

Este bloque protege el acceso y evita enviar un formulario vacío a Firebase.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
045     fun login(email: String, password: String) {
046         if (email.isBlank() || password.isBlank()) {
047             mutableError.value = "Escribe tu correo y contraseña."
048             return
049         }
050         viewModelScope.launch {
051             mutableBusy.value = true
052             mutableError.value = null
053             val result = repository.iniciarSesion(email, password)
054             mutableBusy.value = false
055             if (result.isFailure) mutableError.value = "No se pudo iniciar sesión. Comprueba
     tus credenciales."
056         }
057     }
```

Lectura del bloque (líneas 45-57 del archivo):

Comprobación inicial: isBlank detecta campos vacíos o formados por espacios. El mensaje se guarda en mutableError y return termina la función antes de llamar al repositorio.

viewModelScope.launch: inicia una corrutina vinculada al ViewModel. La espera de Firebase no congela la pantalla.

mutableBusy: informa que hay una solicitud en curso. Se limpia el error anterior y se espera iniciarSesion. Al terminar se libera el formulario.

Resultado: si falla el acceso, se muestra una indicación comprensible. La sesión y el rol correctos se reciben por el flujo de sesión del repositorio.

En la aplicación: Un formulario vacío mantiene al usuario en login. Una contraseña equivocada tampoco abre el catálogo. PA-03 y PA-04 comprueban estos dos comportamientos.

5. Identificación del rol

Archivo: data/auth/FirebaseAuthRepository.kt

Firebase identifica la cuenta; Firestore decide su rol. No se deduce el rol por el texto del correo ni se permite elegirlo desde el login.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
026                 firestore.collection("usuarios").document(user.uid).get()
027                     .addOnSuccessListener { doc ->
028                         if (auth.currentUser?.uid != user.uid) return@addOnSuccessListener
029                         val role = when (doc.getString("rol")) {
030                             "admin" -> Role.ADMIN
031                             "usuario" -> Role.USER
032                             else -> null
033                         }
034                         if (role == null) {
035                             auth.signOut()
036                             trySend(SessionState.Error("Tu cuenta no tiene un rol válido."))
037                         } else {
038                             trySend(SessionState.SignedIn(Session(user.uid,
    user.email.orEmpty(), role)))
039                         }
040                     }
041                     .addOnFailureListener {
```

Lectura del bloque (líneas 26-41 del archivo):

usuarios/{uid}: el documento se busca con el identificador único generado por Firebase. Así se vincula la sesión con el registro correcto.

Comprobación del UID: si la cuenta cambió mientras llegaba la respuesta, se ignora ese resultado. Esto evita aplicar datos de una sesión anterior.

when del campo rol: solo admin y usuario producen roles admitidos. Un documento ausente o un valor distinto no concede acceso.

trySend: publica SignedIn con UID, correo y rol. Ante un rol inválido se cierra la sesión y se comunica el error. Las reglas de Firestore impiden cambiar el rol desde el cliente.

En la aplicación: Las pruebas PA-01 y PA-02 muestran que el panel aparece solamente en la cuenta administradora.

6. Dos consultas reales a TMDB

Archivo: data/remote/MovieApi.kt

Retrofit describe las peticiones como funciones Kotlin. Los resultados se reciben en objetos DTO antes de transformarse al modelo local.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
009 interface MovieApi {
010     @GET("discover/movie")
011     suspend fun discover(@Query("language") language: String = "es-PE", @Query("sort_by")
    sort: String = "popularity.desc", @Query("page") page: Int = 1): MoviePageDto
012
013     @GET("movie/{id}")
014     suspend fun detail(@Path("id") id: Int, @Query("language") language: String = "es-PE"):
    MovieDto
```

Lectura del bloque (líneas 9-14 del archivo):

@GET("discover/movie"): consulta la primera página de películas populares. @Query añade idioma, orden remoto y página; el resultado MoviePageDto contiene la colección results.

@GET("movie/{id}"): consulta una película concreta. @Path reemplaza {id} con el identificador recibido desde la navegación.

suspend: permite esperar la respuesta en una corrutina. No significa ejecutar en el hilo visual una espera bloqueante.

Separación: RetrofitClient configura la conexión; el repositorio decide cuándo consultar y cómo guardar. La búsqueda visible del catálogo filtra Room; no se presenta como búsqueda remota.

En la aplicación: La prueba de detalle registra una llamada /3/movie/ID. La traza guarda únicamente la ruta, sin exponer la clave de TMDB.

7. Room como fuente de la interfaz

Archivo: data/local/MovieDao.kt

El DAO reúne las consultas de SQLite. La interfaz observa los datos que ya están almacenados, tanto con Internet como sin él.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
009 @Dao
010 interface MovieDao {
011     @Query("SELECT * FROM movies ORDER BY rating DESC, title ASC")
012     fun observeAll(): Flow<List<MovieEntity>>
013 
014     @Query("SELECT * FROM movies WHERE favorite = 1 ORDER BY title ASC")
015     fun observeFavorites(): Flow<List<MovieEntity>>
016 
017     @Query("SELECT * FROM movies WHERE id = :id LIMIT 1")
018     fun observe(id: Int): Flow<MovieEntity?>
019 
020     @Query("SELECT * FROM movies WHERE id = :id LIMIT 1")
021     suspend fun get(id: Int): MovieEntity?
022 
023     @Query("SELECT * FROM movies")
024     suspend fun getAll(): List<MovieEntity>
025 
026     @Insert(onConflict = OnConflictStrategy.REPLACE)
027     suspend fun upsert(items: List<MovieEntity>)
```

Lectura del bloque (líneas 9-27 del archivo):

observeAll: selecciona todas las películas y devuelve Flow. Room emite otra lista al cambiar la tabla.

observeFavorites: filtra favorite = 1 y ordena por título. La pantalla Favoritos utiliza esta consulta para mostrar el estado persistente.

observe y get: ambas consultan un ID; la primera sigue los cambios y la segunda devuelve una lectura puntual para una operación del repositorio.

upsert: inserta o reemplaza registros por clave. Por eso el repositorio recupera los favoritos anteriores antes de reemplazar una película.

En la aplicación: Marcar el corazón actualiza Room. Al cerrar y volver a abrir, el favorito sigue presente: no dependía de una variable temporal de la pantalla.

8. Sincronizar conservando favoritos

Archivo: data/repository/MovieRepositoryImpl.kt

La descarga debe actualizar la información remota sin borrar las decisiones locales del usuario.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
023     override suspend fun refrescar(): Result<Int> = capturar {
024         val remote = api.discover().results
025         database.withTransaction {
026             val local = dao.getAll().associateBy { it.id }
027             val updated = remote.map { dto ->
028                 dto.toEntity(
029                     favorite = local[dto.id]?.favorite ?: false,
030                     detailLoaded = local[dto.id]?.detailLoaded ?: false
031                 ).let { entity ->
032                     val old = local[dto.id]
033                     if (old?.detailLoaded == true) entity.copy(overview = old.overview) else
     entity
034                 }
035             }
036             dao.upsert(updated)
037         }
038         remote.size
039     }
```

Lectura del bloque (líneas 23-39 del archivo):

remote: contiene las películas recibidas de TMDB. La descarga ocurre antes de abrir la transacción de base de datos.

withTransaction: agrupa las lecturas y escrituras locales. associateBy crea un mapa por ID para encontrar el registro previo de cada película.

toEntity: transforma cada DTO y conserva favorite y detailLoaded. Si existía un detalle más completo, también conserva su sinopsis.

dao.upsert: guarda los registros resultantes. remote.size devuelve cuántos llegaron en esta sincronización para construir el mensaje del aviso.

En la aplicación: PA-10 marca una película, sincroniza y vuelve a Favoritos: el corazón se mantiene. La red no sobrescribe la preferencia local.

9. Responder a un fallo de conexión

Archivo: ui/lista/MovieListViewModel.kt

El ViewModel diferencia una descarga correcta de una pérdida de Internet y conserva lo que todavía se puede consultar.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
058     fun sync() {
059         viewModelScope.launch {
060             mutable.update { it.copy(loading = true, error = null, syncCount = null) }
061             try {
062                 val count = repository.refrescar().getOrThrow()
063                 mutable.update { it.copy(loading = false, offline = false, syncCount =
    count) }
064                 notificador.mostrar("Datos actualizados", "Se guardaron $count elementos")
065             } catch (e: kotlinx.coroutines.CancellationException) {
066                 throw e
067             } catch (e: Exception) {
068                 mutable.update {
069                     it.copy(
070                         loading = false,
071                         offline = it.movies.isNotEmpty(),
072                         error = if (it.movies.isEmpty()) "No se pudo cargar el catálogo.
    Revisa la conexión y vuelve a intentar." else null
073                     )
074                 }
075             }
076         }
077     }
```

Lectura del bloque (líneas 58-77 del archivo):

Inicio: loading pasa a true y se limpian los datos de error de la operación anterior. refrescar().getOrThrow transforma un Result fallido en una excepción controlada.

Éxito: loading se desactiva, se elimina el aviso offline y se informa cuántos registros se guardaron. Solo entonces se solicita la notificación.

Cancelación: CancellationException se vuelve a lanzar para respetar la finalización de la corrutina.

Fallo: con películas locales se activa offline y se mantiene la lista; sin películas se genera un error que la UI muestra con Reintentar. El repositorio contempla IOException y HttpException.

En la aplicación: En modo avión con datos, PA-08 muestra el catálogo y un aviso. Con Room vacío, PA-09 muestra el error; al recuperar la red el botón permite descargar.

10. Reintento en una instalación nueva

Archivo: ui/login/AuthViewModel.kt

Una instalación sin sesión y sin red no puede autenticar al usuario. Se muestra un error recuperable desde el arranque, sin intentar saltarse el acceso.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
032     private var observacion: kotlinx.coroutines.Job? = null
033
034     init { reintentarSesion() }
035
036     /** Vuelve a consultar la sesión y la conexión sin duplicar el listener de Firebase. */
037     fun reintentarSesion() {
038         observacion?.cancel()
039         mutableSession.value = SessionState.Loading
040         observacion = viewModelScope.launch {
041             repository.session.collect { mutableSession.value = it }
042         }
043     }
```

Lectura del bloque (líneas 32-43 del archivo):

observacion: conserva el trabajo que escucha los cambios de sesión. init inicia la primera observación.

cancel: antes de reintentar termina la observación anterior. El repositorio elimina su listener al cancelarse el flujo.

Cargando: informa al usuario de la consulta. La nueva corrutina vuelve a observar el repositorio, que comprueba si ya hay conexión.

Recuperación: al existir red y no existir una sesión, se publica SignedOut y reaparece el formulario. La prueba unitaria comprueba que solo quede un observador activo.

En la aplicación: PA-09 se repite con los datos de la aplicación limpiados y modo avión: aparece Reintentar. Después de activar Internet vuelve al login.

11. Notificar sin bloquear la app

Archivo: notification/NotificadorAndroid.kt

La interfaz Notificador permite pedir un aviso desde el ViewModel sin introducir Context en esa capa.

Extracto del archivo real; se omiten imports y otros métodos para concentrar la explicación.

```
011 class NotificadorAndroid(private val context: Context) : Notificador {
012     private var pendiente: Pair<String, String>? = null
013
014     override fun mostrar(titulo: String, texto: String) {
015         if (puedeNotificar()) SyncNotifications.show(context, titulo, texto)
016         else pendiente = titulo to texto
017     }
018
019     fun permisoActualizado(concedido: Boolean) {
020         val aviso = pendiente ?: return
021         pendiente = null
022         if (concedido) SyncNotifications.show(context, aviso.first, aviso.second)
023     }
```

Lectura del bloque (líneas 11-23 del archivo):

mostrar: comprueba el permiso. Si ya está concedido, delega en SyncNotifications; si Android todavía espera la decisión, conserva temporalmente el aviso.

permisoActualizado: recupera el mensaje pendiente y lo borra para no repetirlo. Solo lo muestra cuando el usuario concedió el permiso.

Implementación Android: SyncNotifications crea el canal catalog_sync y usa PendingIntent para abrir MainActivity al tocar el aviso.

Independencia: denegar el permiso no cancela la descarga ni la escritura en Room. La operación principal no depende de publicar una notificación.

En la aplicación: PA-05 comprueba el texto con la cantidad y la apertura de la app al tocarlo. PA-14 deniega el permiso y verifica que el catálogo continúa funcionando.

12. Estados, navegación y rotación

```
003 sealed interface UiState<out T> {
004     data object Cargando : UiState<Nothing>
005     data class Exito<T>(val datos: T) : UiState<T>
006     data class Error(val mensaje: String) : UiState<Nothing>
007 }
```

UiState expresa tres situaciones: una operación en curso, datos disponibles o un error recuperable. Los ViewModel publican StateFlow y las pantallas los observan con collectAsStateWithLifecycle. Los estados especializados conservan datos como el filtro, el orden o el aviso offline.

AppNav.kt reúne las rutas y recibe dependencias mediante AppContainer. Las pantallas envían eventos al ViewModel y no realizan consultas HTTP ni consultas Room directamente.

Rotación: el ViewModel conserva el filtro y no vuelve a ejecutar su inicialización al recrearse la actividad. CineApplication mantiene el contenedor durante el proceso. El encabezado del catálogo forma parte de la misma LazyColumn que las tarjetas: puede desplazarse en horizontal sin quitarles toda la altura.

En PA-13 se escribió Coco, se giró el emulador y se comprobó que la tarjeta seguía accesible al desplazar. La traza HTTP mantuvo el mismo número de solicitudes discover antes y después del giro.

13. Pruebas automatizadas

Gradle compiló el APK y ejecutó 8 pruebas sin fallos: 7 del ViewModel de catálogo y 1 del ViewModel de acceso. Los repositorios falsos simulan resultados para que estas pruebas no dependan de una conexión real.

• Carga correcta de películas y cantidad sincronizada.

• Fallo de red sin caché: error recuperable.

• Fallo de red con caché: lista conservada y aviso offline.

• Búsqueda local por título y resultado vacío.

• Orden por valoración compatible con el filtro de favoritas.

• Lectura del estado sin disparar una descarga adicional.

• Notificación con cantidad solo cuando la sincronización funciona.

• Reintento del acceso al recuperar red sin duplicar observadores.

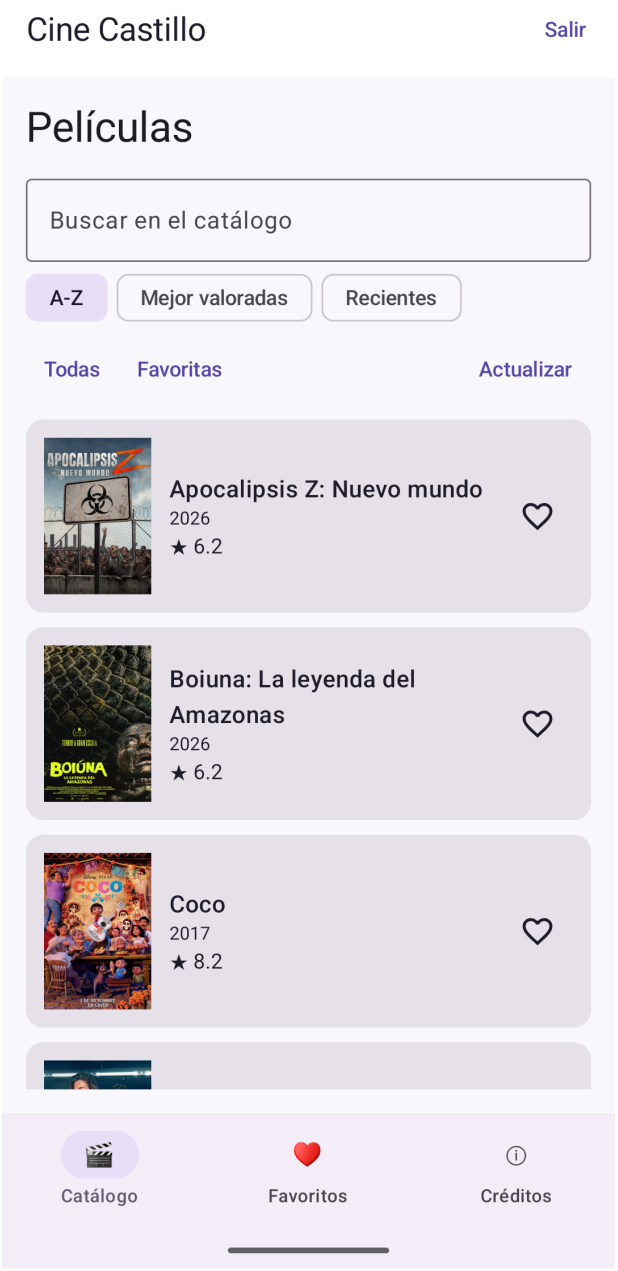
14. Registro de aceptación en el emulador

Dispositivo: Pixel 8 con Google Play, Android API 37.2. Ejecución: 11/11/2026. Se utilizaron una cuenta usuario y otra admin, cambios reales de conectividad, cierre del proceso, rotación y decisiones reales del permiso. Las capturas no son diseños ni imágenes generadas.

Caso	Resultado observado
PA01 **Aprobado**	Cuenta usuario accede al catálogo y no tiene opción Admin.
PA02 **Aprobado**	La cuenta admin tiene acceso al catálogo y al panel administrativo.
PA03 **Aprobado**	Contraseña incorrecta rechazada con mensaje claro.
PA04 **Aprobado**	Campos vacíos rechazados antes de acceder a Firebase.
PA05 **Aprobado**	Primera sesión con Room vacío: carga, catálogo de 20 películas y notificación. Al pulsar el aviso se abre la app.
PA06 **Aprobado**	Detalle visible y llamada real al segundo endpoint registrada sin credenciales en la traza.
PA07 **Aprobado**	La película favorita sigue guardada después de detener y reabrir la app.
PA08 **Aprobado**	En modo avión se ven las películas de Room y el aviso sin conexión.
PA09 **Aprobado**	Sin datos ni sesión (datos de app limpiados), en modo avión: error con Reintentar. Al activar Internet vuelve al login. También se verificó error y recuperación del catálogo con Room vacío.
PA10 **Aprobado**	El favorito permanece después de sincronizar el catálogo.
PA11 **Aprobado**	Vaciar deja 0 películas y 0 favoritas; reintentar con red recupera 20 películas y el panel muestra los nuevos totales.
PA12 **Aprobado**	Al cerrar sesión y reabrir se exige autenticación.
PA13 **Aprobado**	Filtro y contenido conservados al girar. La tarjeta es accesible mediante scroll. Traza HTTP confirma cero descargas nuevas por rotación.
PA14 **Aprobado**	Permiso denegado en el diálogo: la sincronización y el catálogo funcionan, sin aviso de Cine Castillo en el panel de notificaciones.

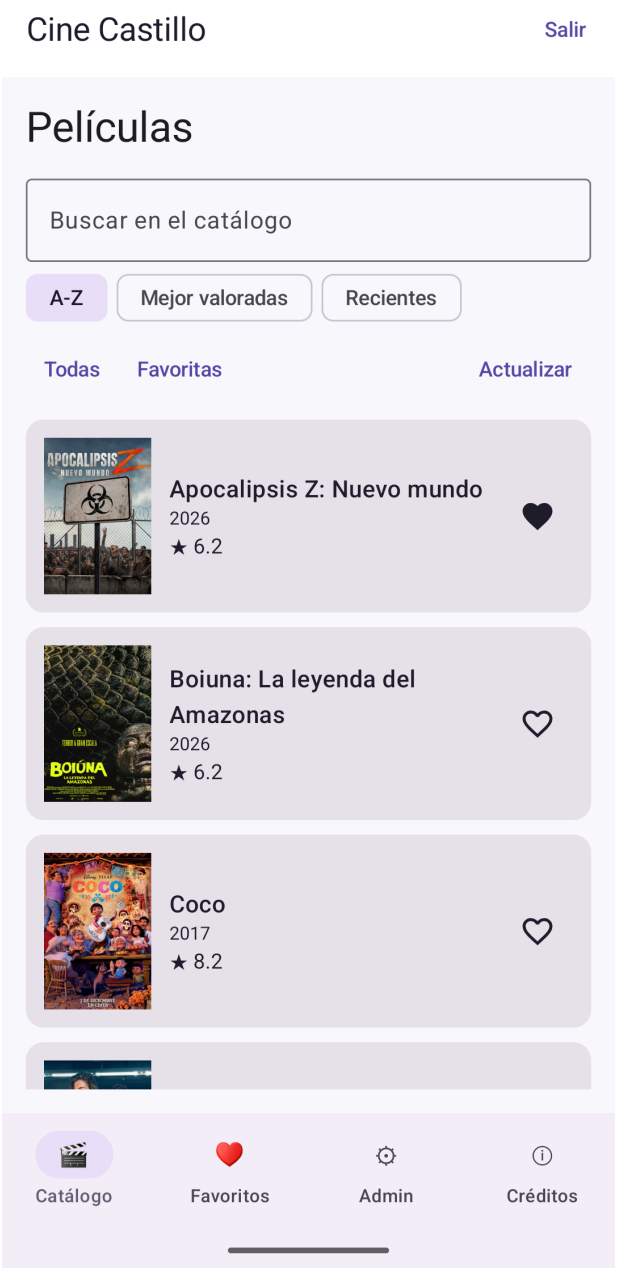
15. Evidencias de funcionamiento

PA-01. Acceso del usuario



Catálogo visible sin entrada al panel Admin.

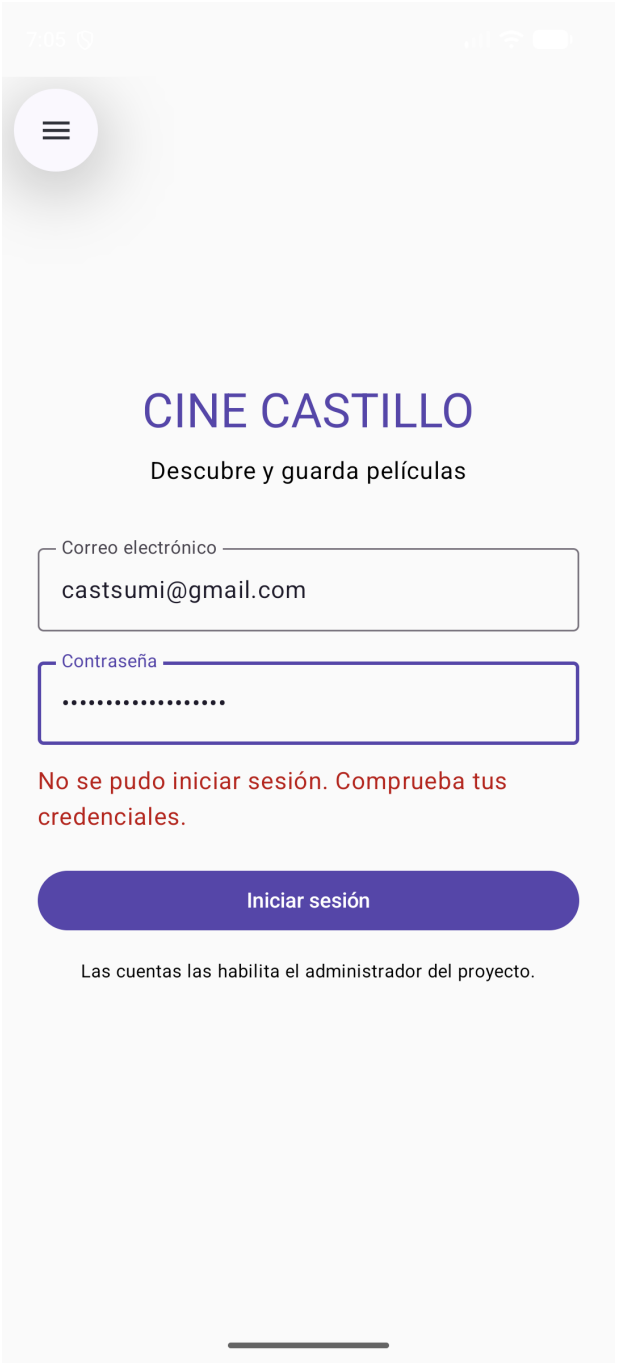
PA-02. Acceso del administrador



El mismo catálogo incorpora la opción Admin según el rol leído de Firestore.

15. Evidencias de funcionamiento

PA-03. Contraseña incorrecta



El acceso se rechaza y se explica que deben comprobarse las credenciales.

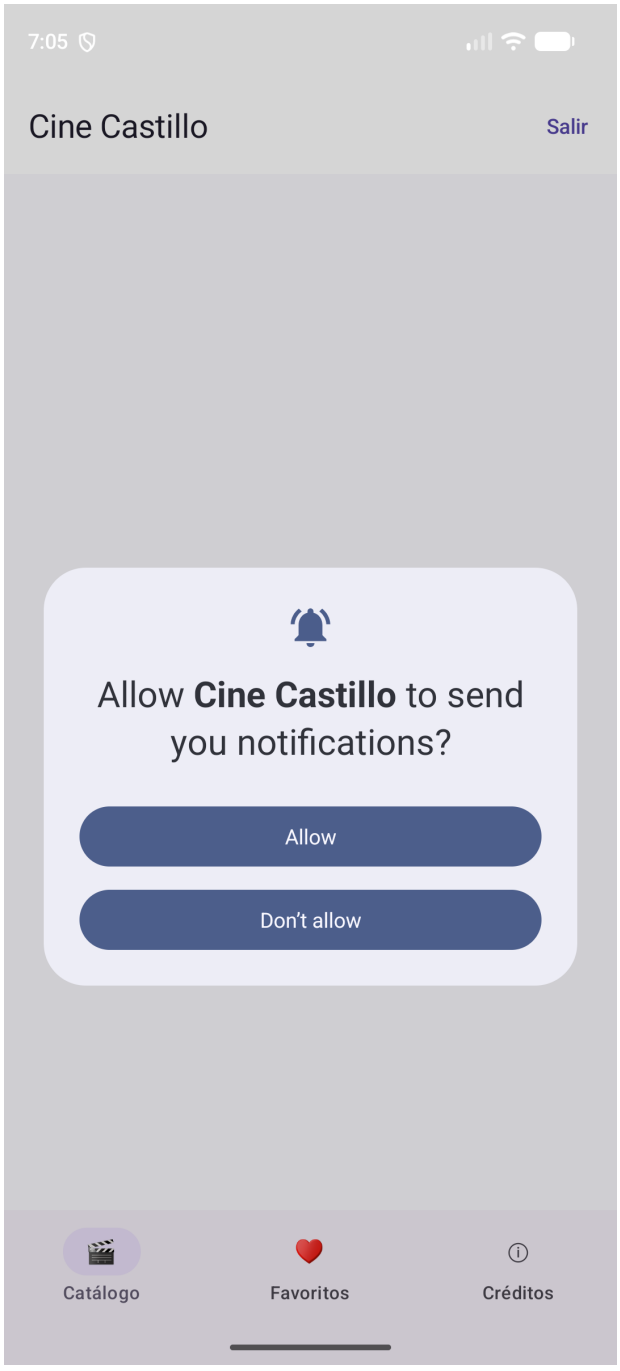
PA-04. Formulario vacío



Se solicita completar correo y contraseña antes de autenticar.

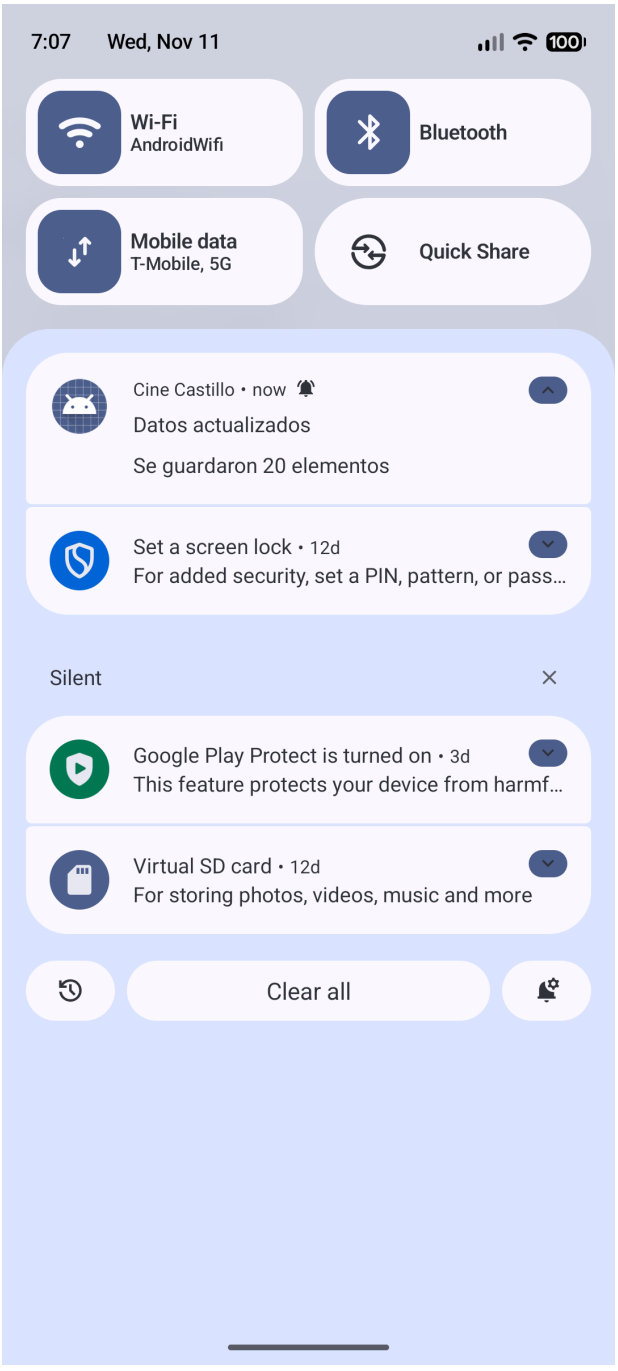
15. Evidencias de funcionamiento

PA-05. Carga inicial



Estado de carga entre el acceso y la primera lista de películas.

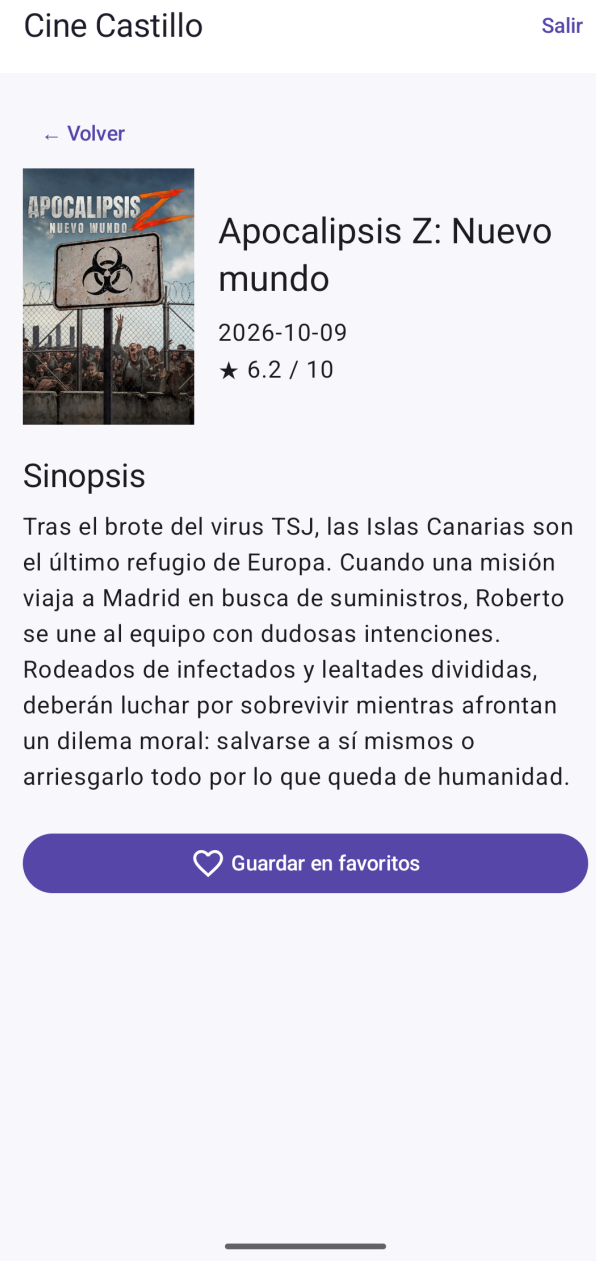
PA-05. Notificación



El aviso informa la cantidad guardada. Se comprobó la apertura de la app al tocarlo.

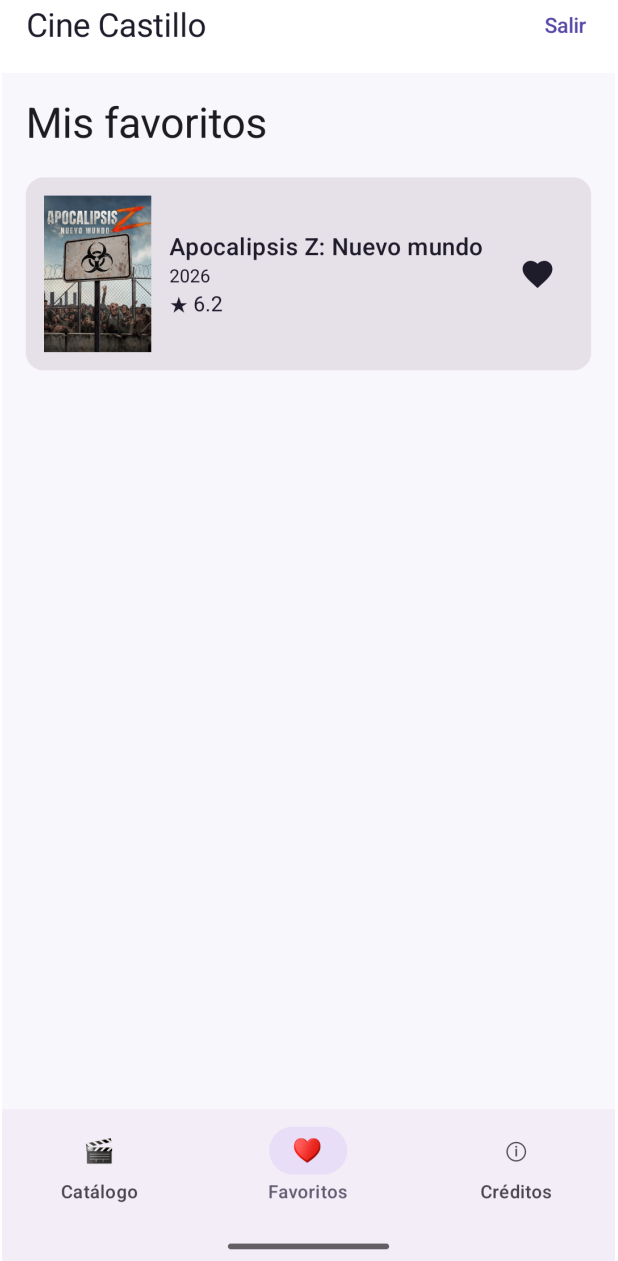
15. Evidencias de funcionamiento

PA-06. Detalle



Datos de una película obtenidos mediante el endpoint por ID.

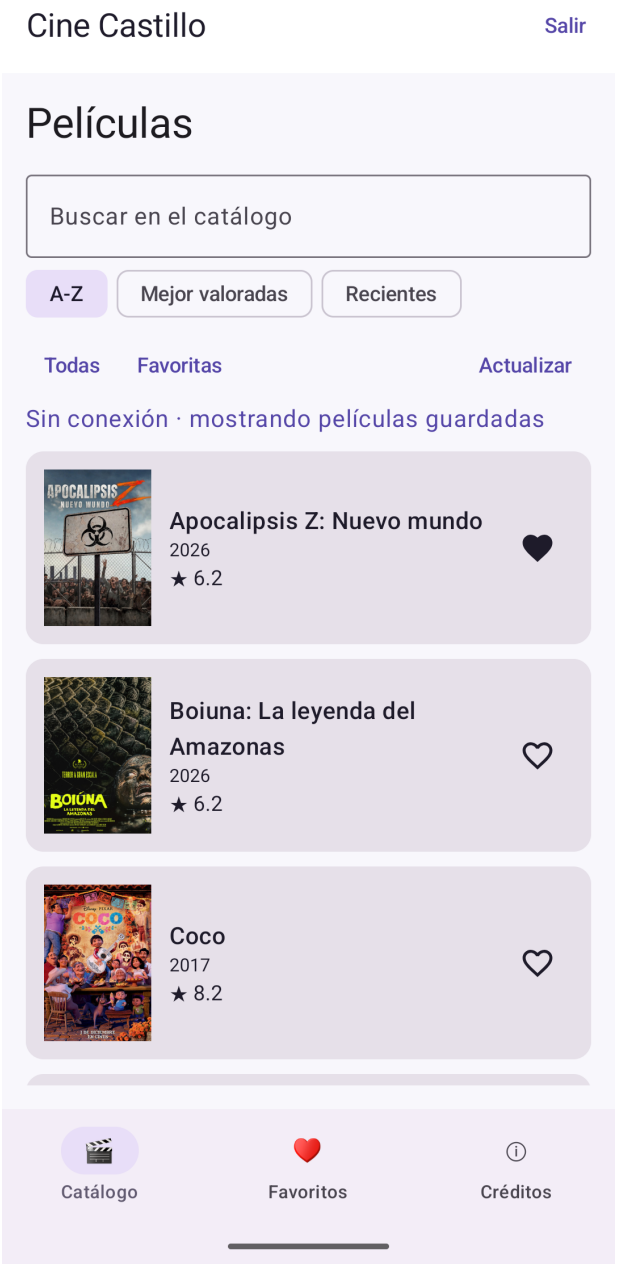
PA-07. Persistencia



Favorito conservado después de cerrar el proceso y reabrir.

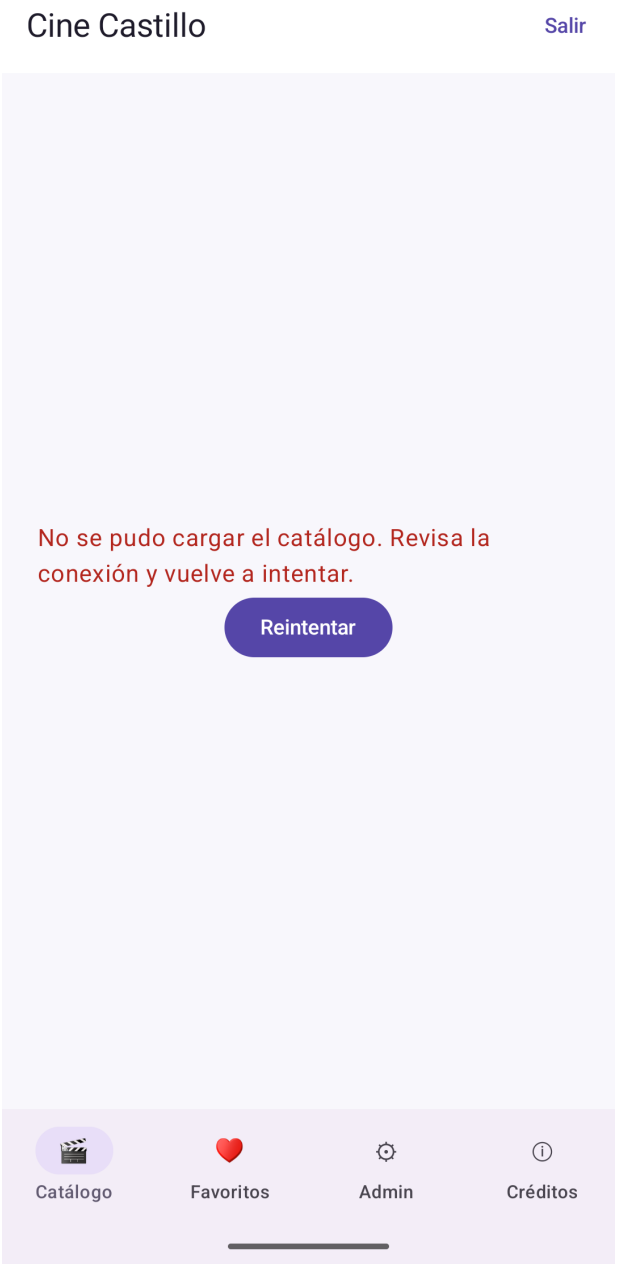
15. Evidencias de funcionamiento

PA-08. Sin red, con caché



Las películas locales siguen disponibles con el aviso sin conexión.

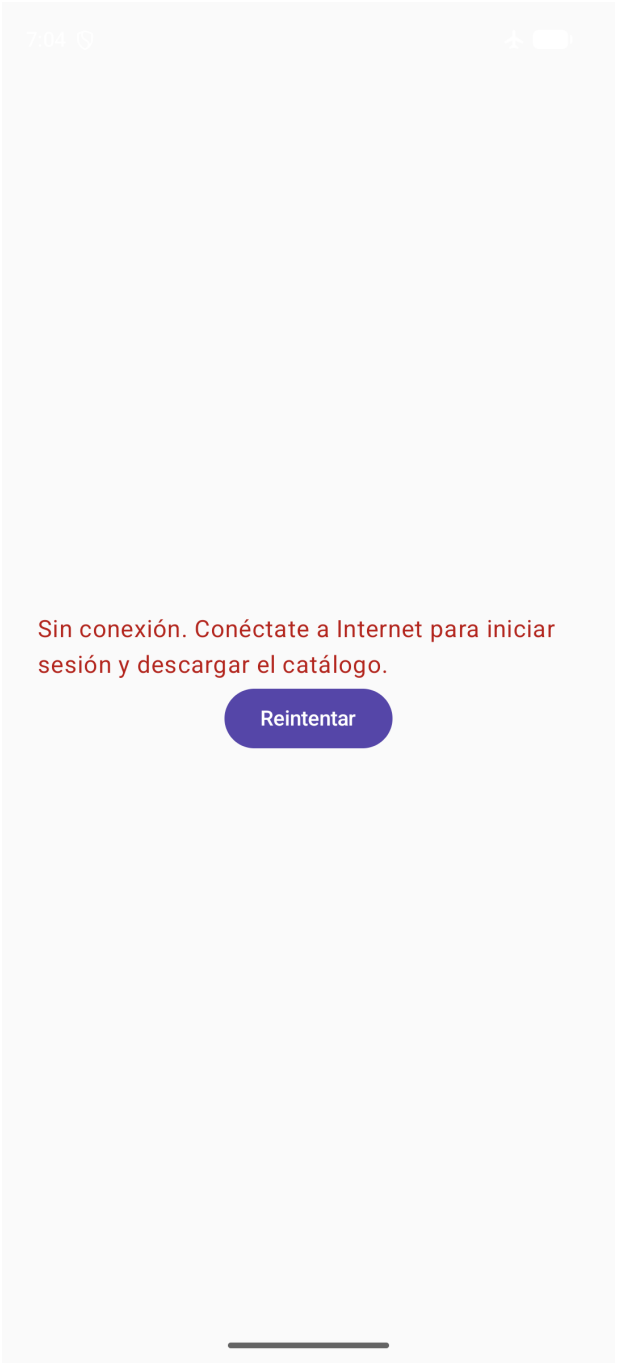
PA-09. Sin red, sin caché



Error con Reintentar después de vaciar Room.

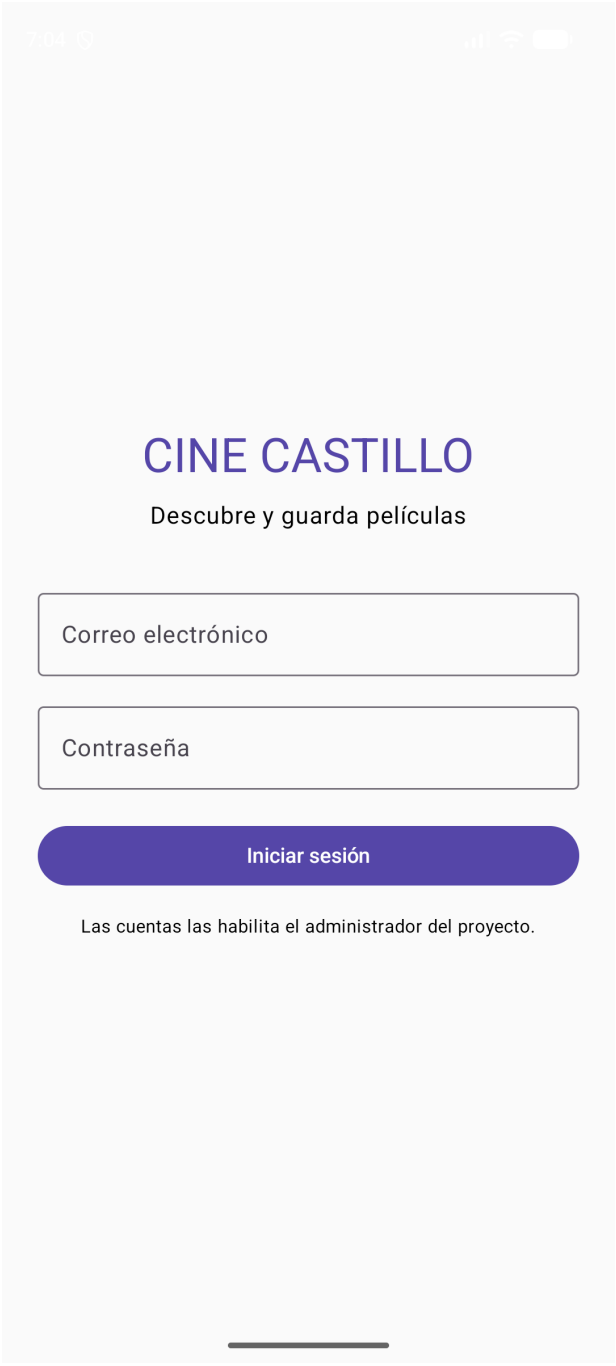
15. Evidencias de funcionamiento

PA-09. Instalación sin datos



El arranque sin sesión ni datos y en modo avión también permite reintentar.

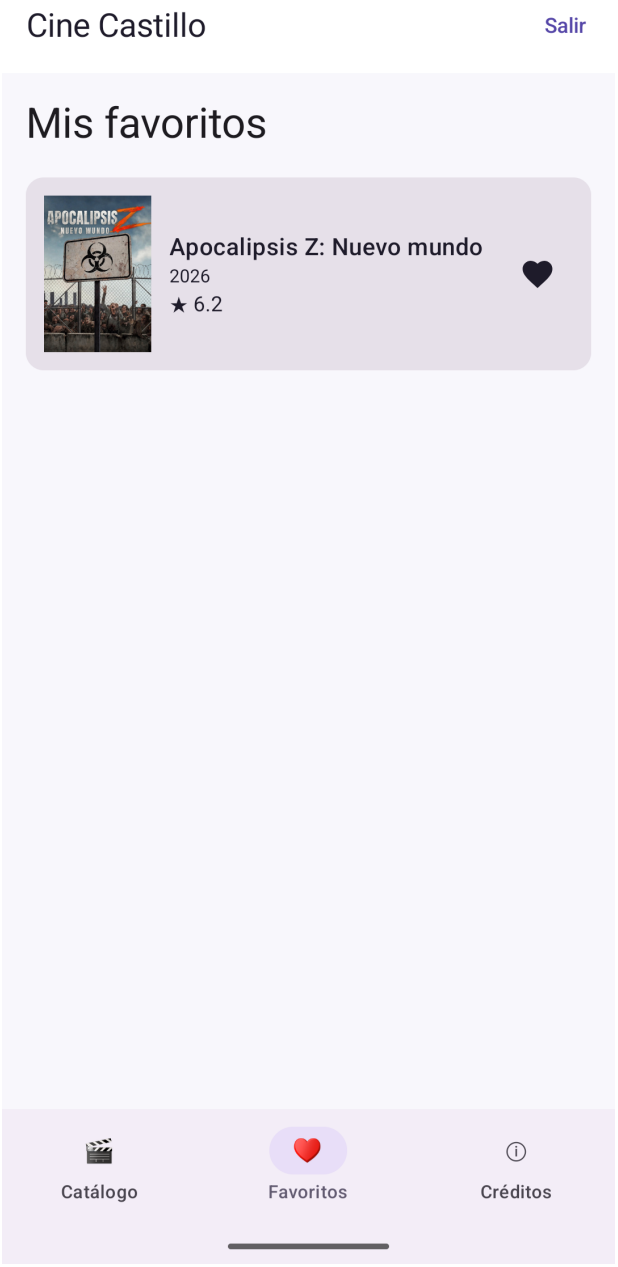
PA-09. Recuperación del acceso



Al activar Internet y reintentar se muestra el formulario de acceso.

15. Evidencias de funcionamiento

PA-10. Favorito sincronizado



Actualizar los datos remotos no elimina la preferencia local.

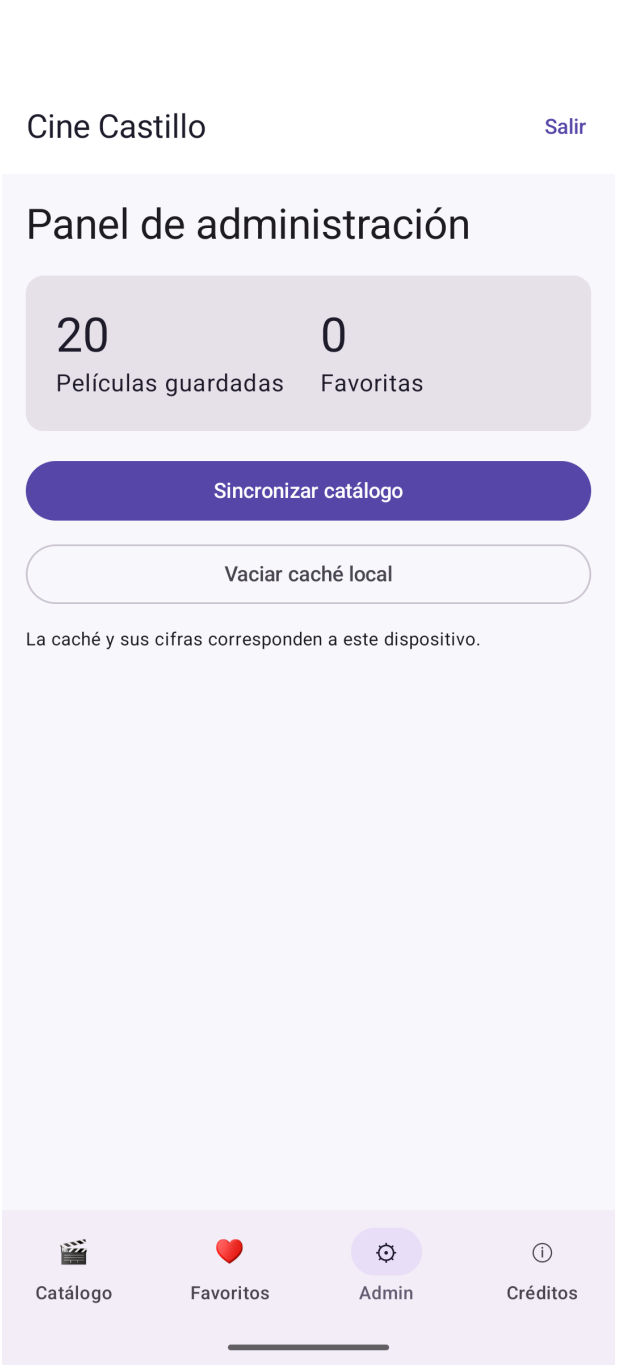
PA-11. Caché vacía



El panel refleja cero películas y cero favoritos después de confirmar.

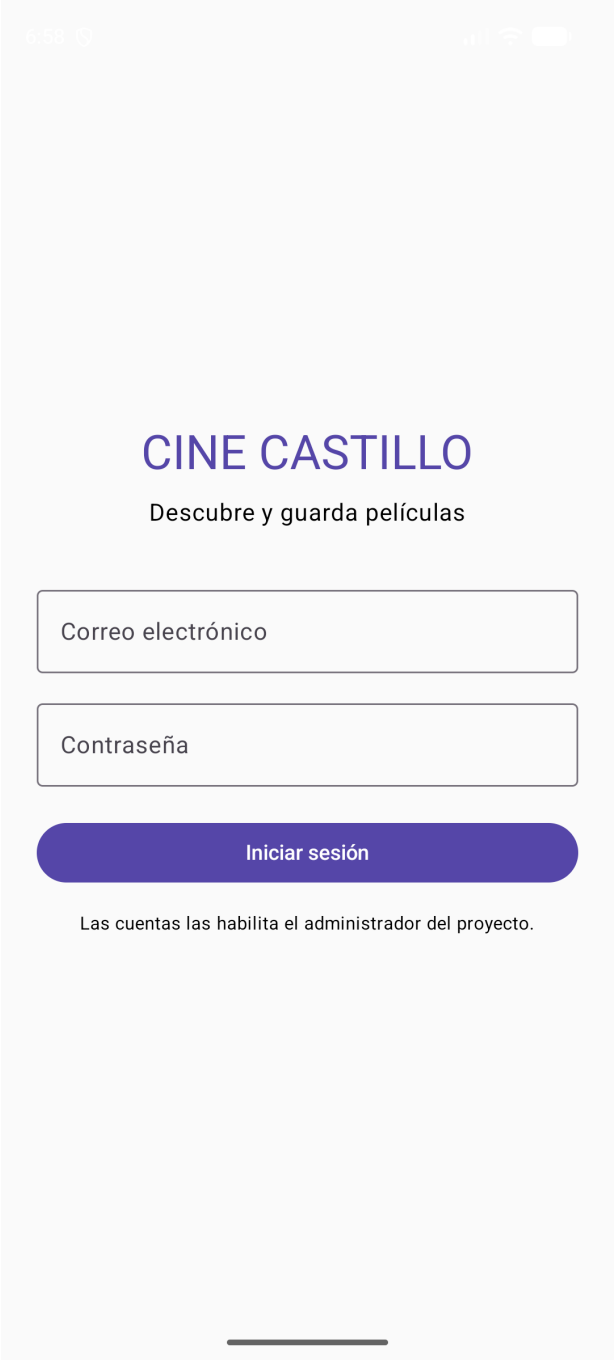
15. Evidencias de funcionamiento

PA-11. Datos recuperados



Al sincronizar vuelven las películas y se actualizan los contadores.

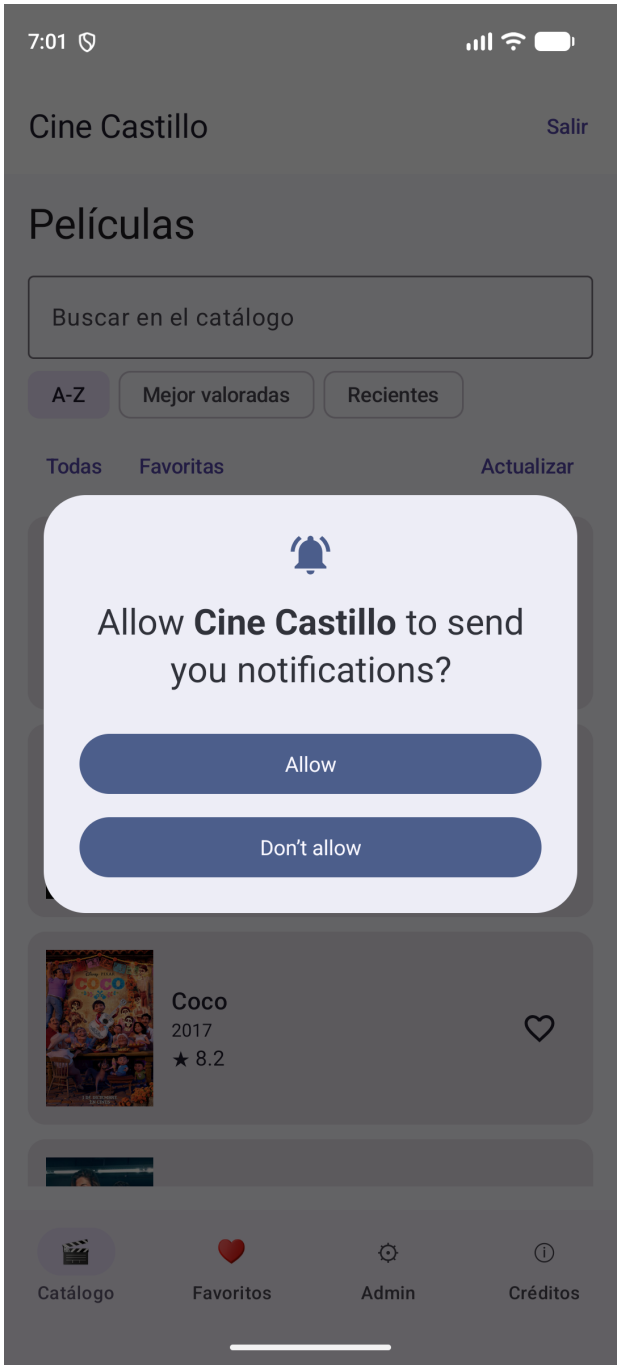
PA-12. Sesión cerrada



Después de salir y reabrir, la aplicación solicita credenciales.

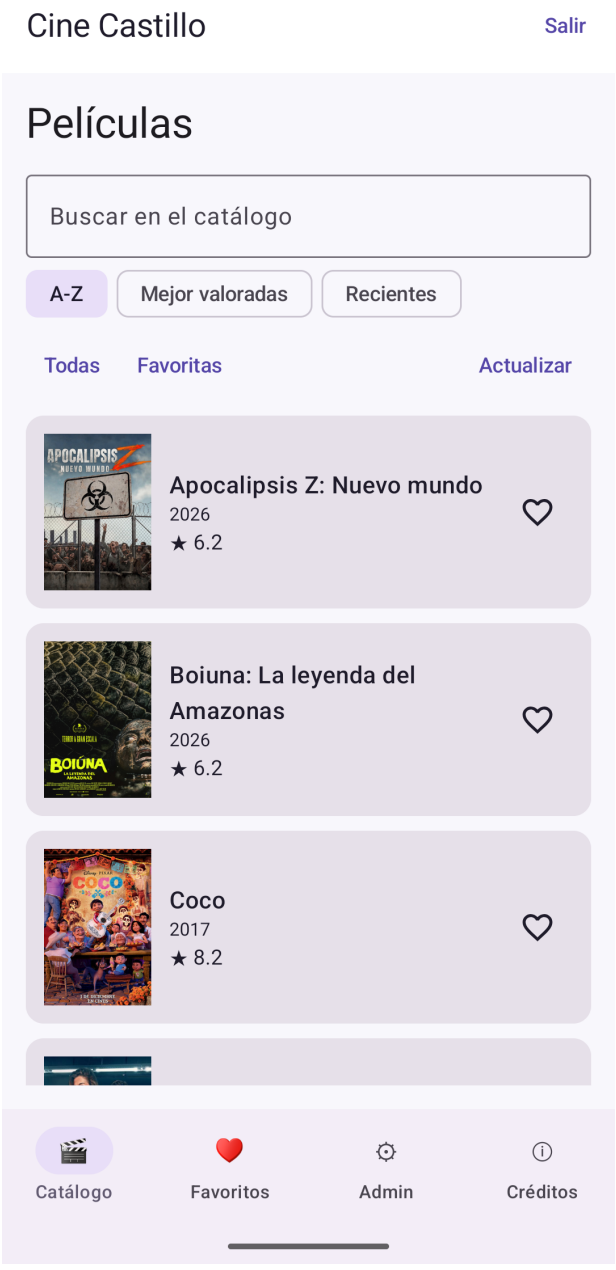
15. Evidencias de funcionamiento

PA-14. Decisión del permiso



Se seleccionó la opción de no permitir notificaciones.

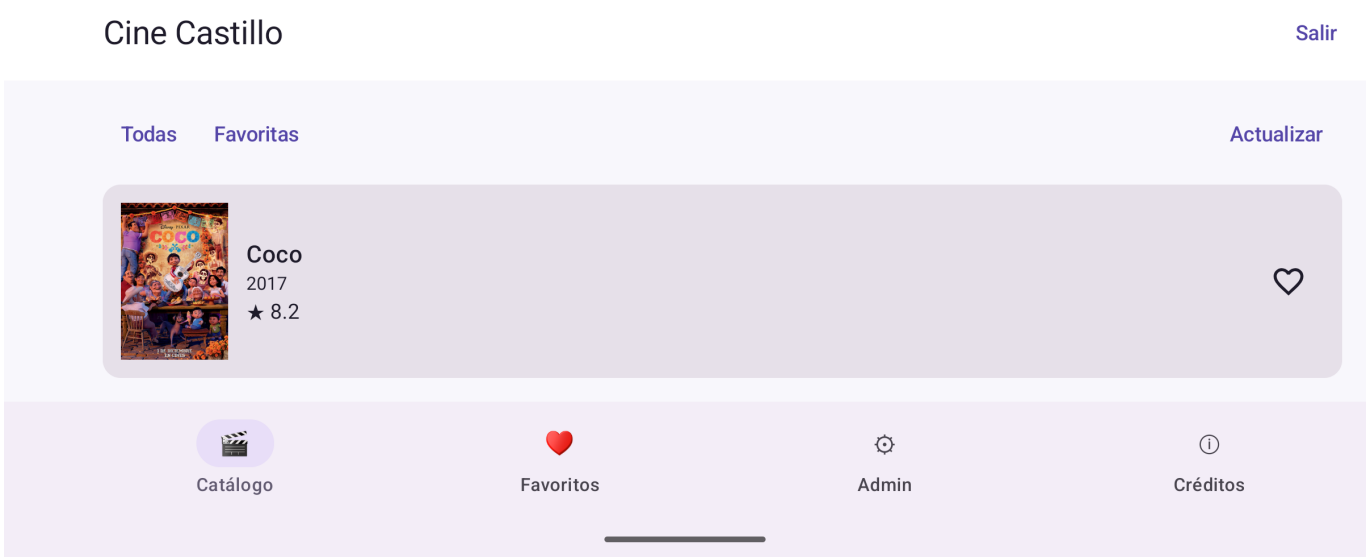
PA-14. Operación sin permiso



La sincronización y el catálogo funcionan aunque el aviso esté denegado.

15. Evidencias de funcionamiento

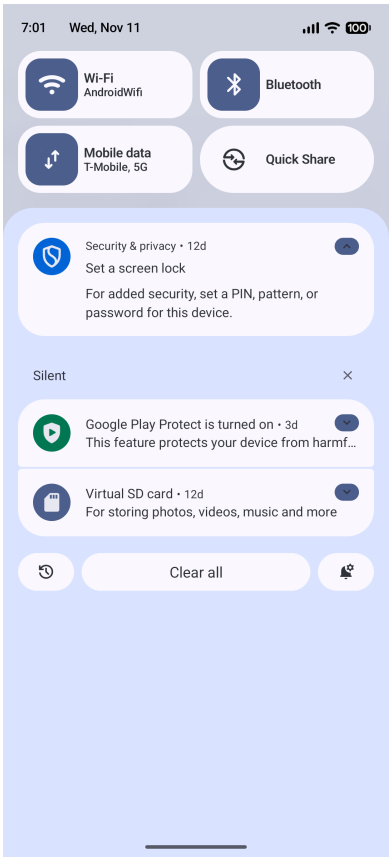
PA-13. Estado conservado al girar



La película Coco continúa accesible en horizontal. El encabezado y las tarjetas comparten el desplazamiento de la lista. Se conservó el filtro y se comprobó el contador de solicitudes HTTP antes y después de la rotación.

GET discover antes=7; después=7. Filtro Coco conservado y tarjeta visible en horizontal.

PA-14. Ausencia de la notificación denegada



La bandeja del sistema no muestra un aviso de sincronización de Cine Castillo después de actualizar con el permiso denegado.

16. Correspondencia con la guía

R01, R02 y R11 - Acceso y sesión: Validación, autenticación Firebase, roles Firestore, persistencia y cierre comprobados con las dos cuentas.

R03 y R04 - Consumo remoto: Catálogo y detalle consultan dos endpoints distintos. Se muestran carga, error con reintento y lista vacía.

R05 - MVVM: Repositorios mediante interfaces, estados observables, ViewModel por función, navegación separada y dependencias suministradas por AppContainer.

R06, R07 y R08 - Datos locales: Room almacena películas y favoritas. Se verificaron persistencia, conservación al sincronizar y recuperación offline con y sin datos.

R09 y R10 - Notificación y administración: Canal y permiso Android, aviso con cantidad, apertura por notificación y panel con sincronización, contadores y vaciado confirmado.

R12 y R13 - Pruebas y mejora: Ocho pruebas unitarias, registro de prompts reales y mejora documentada. Historial de avances separado entre main y mejora-ia.

17. Observaciones

Durante las pruebas comprobé que una compilación correcta no era suficiente: al girar el dispositivo había que asegurar espacio para las tarjetas, y sin conexión había que diferenciar si existían datos guardados. También revisé la notificación mientras Android solicitaba permiso, para que aceptar o rechazar esa opción no afectara la descarga.

Los valores del catálogo son datos externos y pueden cambiar en futuras sincronizaciones. Los conteos de las capturas describen la ejecución realizada. Firebase gestiona la identidad y el rol; Room conserva los datos en el teléfono. Esta separación explica por qué las favoritas permanecen al reabrir en el mismo dispositivo.

18. Conclusión

Con este trabajo logré conectar la interfaz de una aplicación de películas con servicios reales y almacenamiento local. Entendí mejor cómo el repositorio organiza los datos y cómo el ViewModel comunica cada cambio a Compose. Las pruebas me ayudaron a corregir situaciones que no se notan en una ejecución normal, como abrir sin Internet, negar permisos o girar la pantalla. Después de comprobar los 14 casos y las 8 pruebas unitarias, la entrega reúne el código, las evidencias y una explicación del funcionamiento que puedo utilizar en la sustentación.

Repositorio: github.com/JesusCastillo2026/Programacion-En-Moviles
Ubicación: Semana_12/CineCastillo, rama mejora-ia
Versión: 1.0.2
Guía: GLAB-S11-JLEONS-2026 (2).pdf, Laboratorio 12.
Registro de mejoras: PROMPTS.md. **Pruebas:** PRUEBAS_ACEPTACION.md.